Introduction to Computer Science and Programming

Labo 14 – C++ Pointers

Laboratory 14, March 19, 2026

Filippo Ricca - Muhammad Mohsin

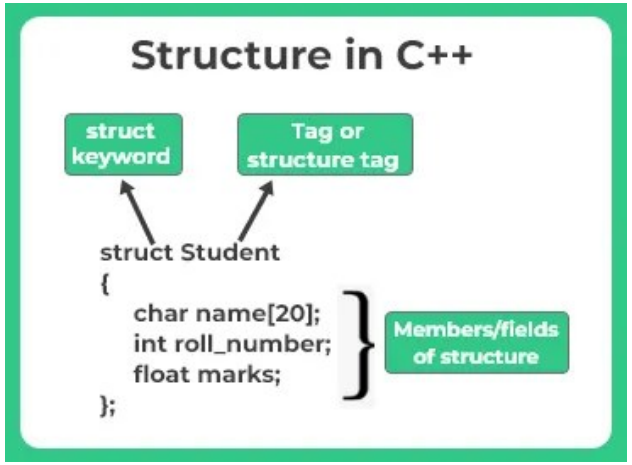
DIBRIS, University of Genova

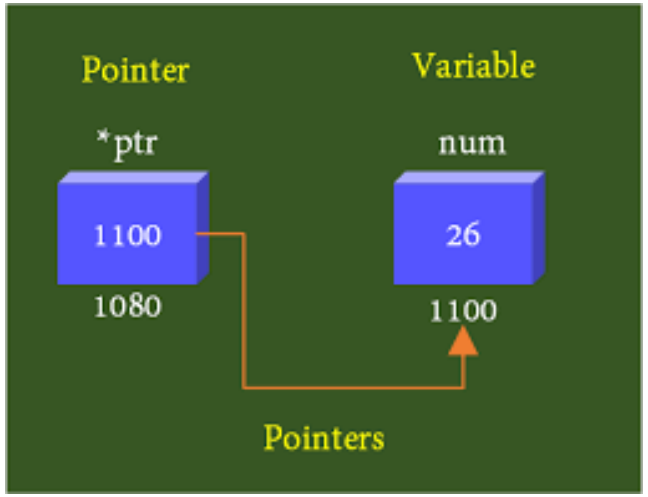
Today…

Today's lab is dedicated to **C++ Pointers**

- Exercise 1: **Matrix Traversal Using Pointers**
- Exercise 2: **String in Reverse Order**
- Exercise 3: **Array Traversal with Pointer Arithmetic**
- Exercise 4: **Swapping Values Using Pointer Parameters**
- Exercise 5: **Array Statistics with const Pointer Parameters**

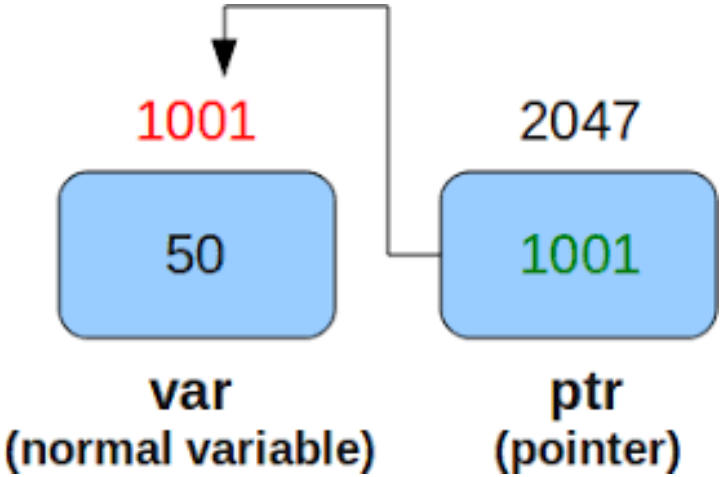
All the C++ programs must be delivered on Aulaweb!





Pointers

- ***Pointer variables***, simply called ***pointers***, are designed to hold memory addresses as their values
- Normally, a variable contains a specific value, e.g., an integer, a floating-point value, and a character
- However, **a pointer contains the memory address of a variable** that in turn contains a specific value



Declaring Pointers

- Type * Varname;
  declares Varname as a pointer to Type

- Examples

```
char* A;
int* B;
struct node * Root;
```

A is a pointer to a character

B is a pointer to an integer

Root is a pointer to a node structure

- * associates with the variable name

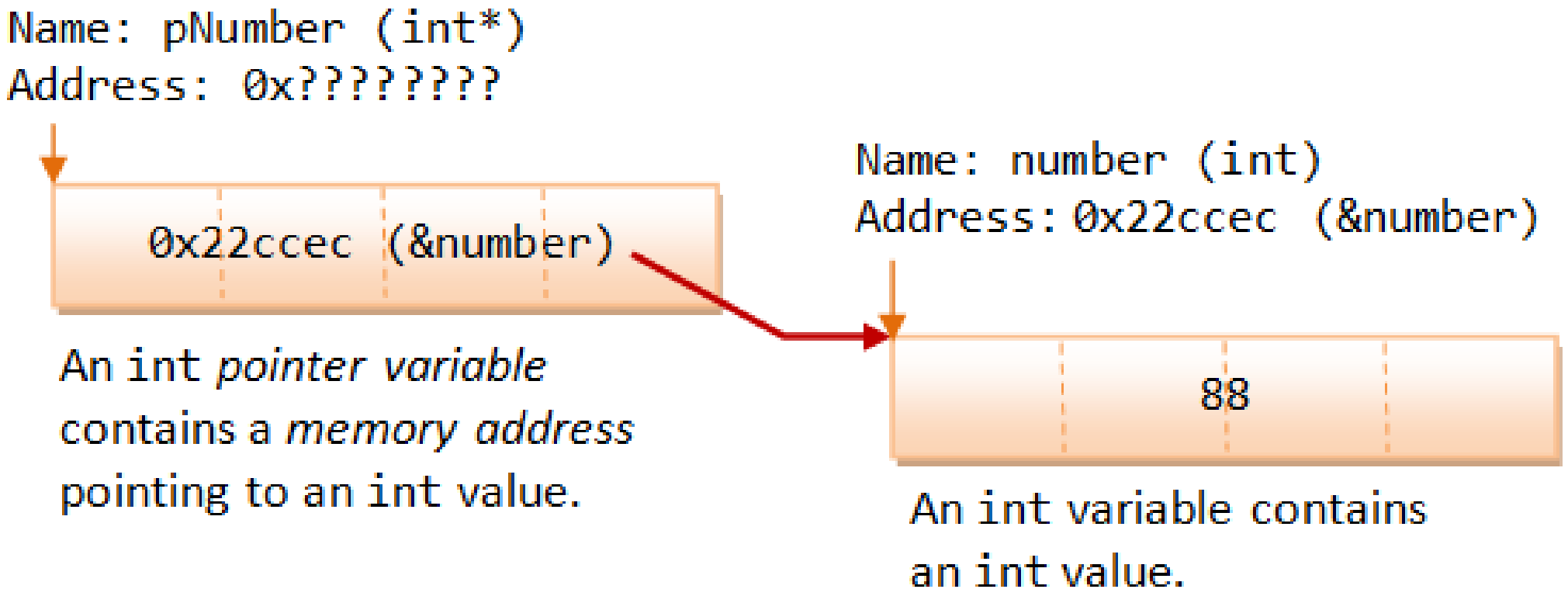
```
char *A, B;
```

A is a pointer to a character

B is a character!

Initializing Pointers via the Address-Of Operator (&)

- When you declare a pointer variable, its content is not initialized. In other words, it contains an address of **"somewhere",** which is of course not a valid location.



Initializing Pointers via the Address-Of Operator (&)

```
int number = 88;      // An int variable with a value
int * pNumber;        // Declare a pointer variable called pNumber pointing to an int
pNumber = &number;    // Assign the address of the variable number to pointer pNumber

int * pAnother = &number; // Declare another int pointer and init to address of the variable number
```

Operators

- At the C++ level, we have two important operators:

- **REFERENCING OPERATOR (&, ampersand)**
  Given a variable x, it returns a reference corresponding to the address of x

```
p = &x;
```

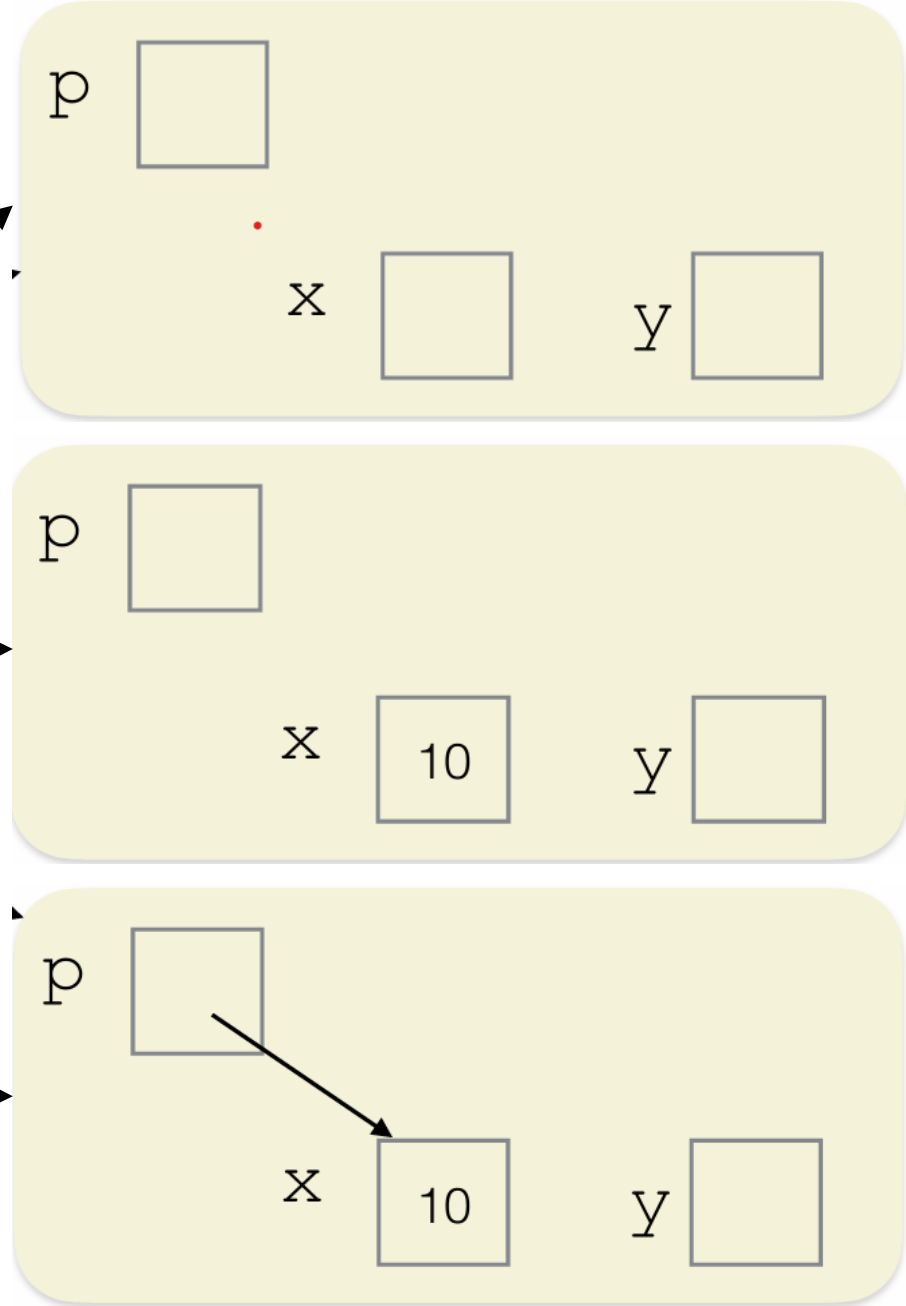
- **DEREFERENCING OPERATOR (*)**
  Given a pointer p, it returns the variable pointed to by p

```
*p
```

Example

```
int x,y;
int *p;

x=10;

p=&x;
```

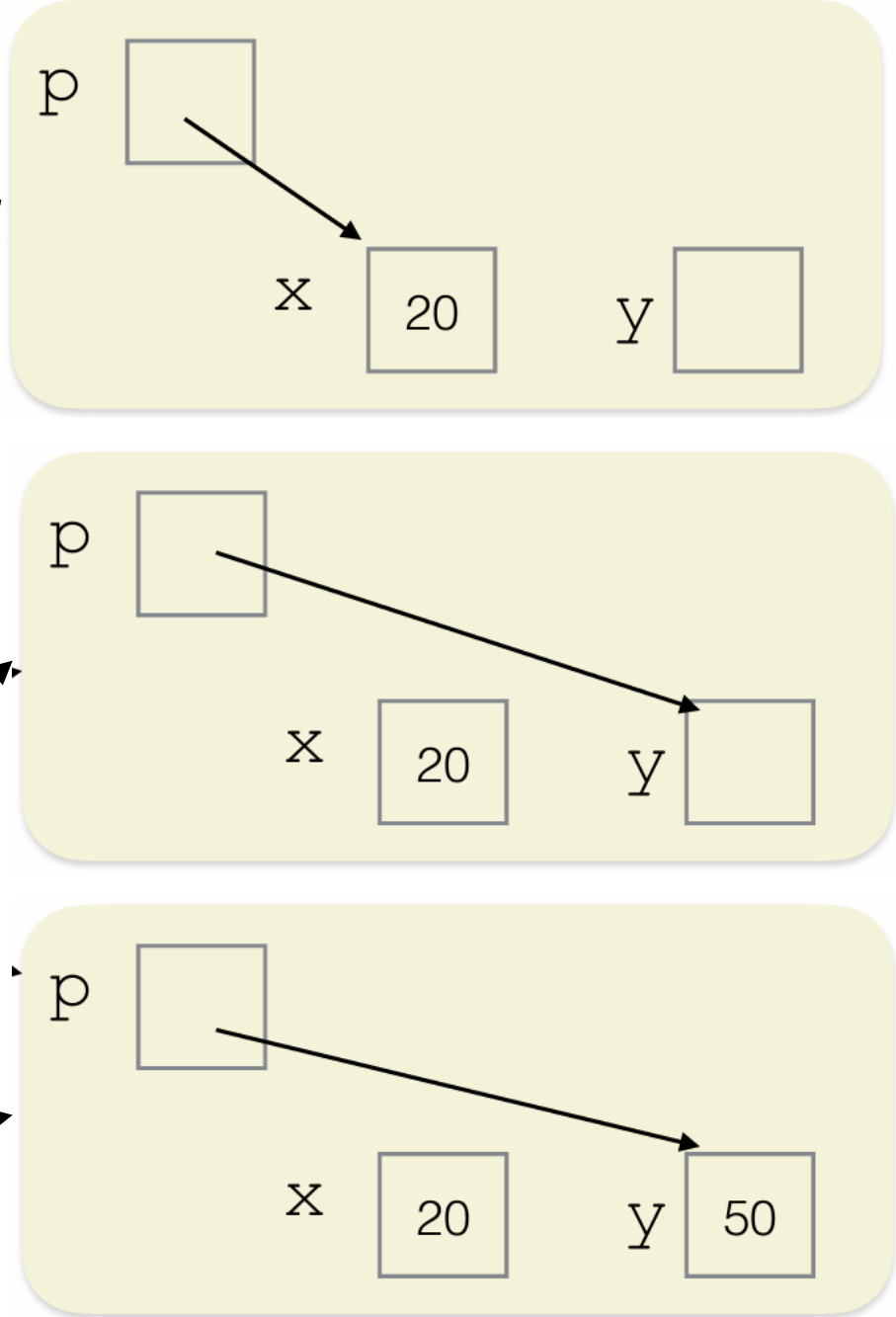
p points to x



PRINT: x 10 y 0 p 0x7fff58030ab8 *p 10

Example

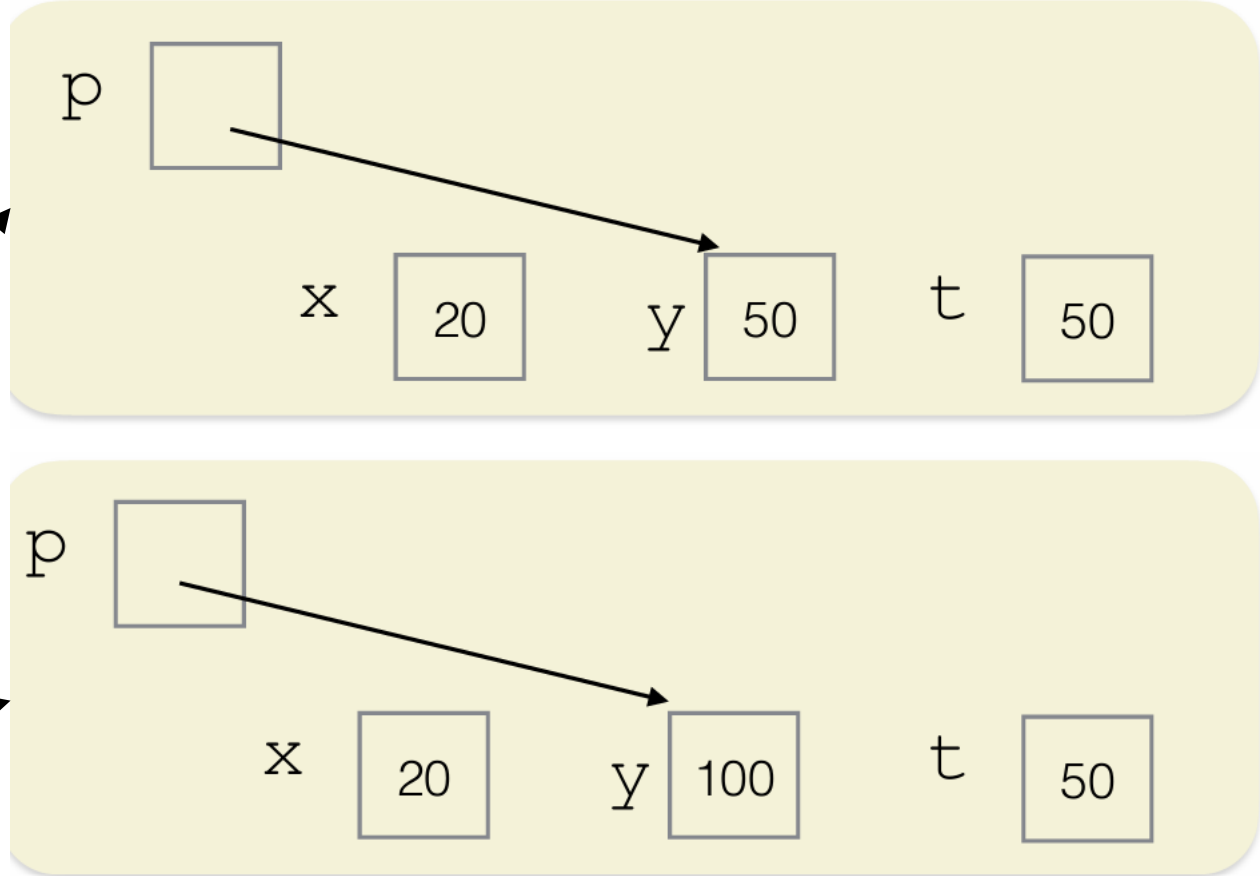
```
int x,y;
int *p;

x=10;

p=&x;

*p = 20;

p = &y;

*p = 50;
```



PRINT: x 20 y 50 p 0x7fff58030ab4 *p 50

Example

```
int x,y;
int *p;

x=10;

p=&x;

*p = 20;

p = &y;

*p = 50;

int t = *p

*p = 100;
```



PRINT: x 20 y 100 p 0x7fff58030ab4 *p 100 t 50

Pay attention!!

- If a pointer points to an **invalid memory address**, the outcome of this operation cannot be predicted
- Either the **program aborts**, or the **results are meaningless**...
  - Ex. Segmentation fault

- How to initialize a pointer?
- If I'm not ready to assign it a real address yet, I can use the null pointer

```
int *p=nullptr;
```

```cpp
#include <iostream>

int main() {
    int* ptr = nullptr;  // Null pointer initialization

    // Dereferencing a null pointer (invalid memory address)
    // This will likely cause the program to abort (crash).
    std::cout << *ptr << std::endl;  // Undefined behavior (null pointer dereference)

    // Alternatively, we can also simulate an invalid pointer with an uninitialized pointer
    int* invalidPtr;  // Uninitialized pointer (invalid memory address)

    // Dereferencing an uninitialized pointer. The result is unpredictable.
    // This could cause a segmentation fault or undefined behavior.
    std::cout << *invalidPtr << std::endl;  // Undefined behavior (uninitialized pointer dere

    return 0;
}
```

Try it!

Exercise 1: Matrix Traversal Using Pointers

Write a C++ program that will create a **2D matrix** with user input and using pointer notation, **print all the elements of matrix**

Requirements:

- Do not use standard array indexing like **arr[i][j]** for printing.
- Use pointers to traverse the matrix.

Hint:

- A 2D array can be accessed using pointers like: ***(*(arr + i) + j)**

Exercise 2: String in Reverse Order

- Write a C++ program to ask **user to enter a string** and print the string in **reverse order** using a pointers.

Requirements:

- Do not use array indexing **(e.g., str[i]).**
- Use pointer arithmetic only.

For example:

If user enter **"CPlusCplusProgrammig"**

The expected output:

"gimmaragroPculpCsulPC"

Exercise 3: Array Traversal with Pointer Arithmetic

- Given the array **int scores[5] = {85, 92, 70, 88, 95}**, write a function **printArray(int* p, int size)** that prints all elements using pointer arithmetic (no array indexing!)
- Write a second function **doubleValues(int* p, int size)** that multiplies each element by 2 using pointer arithmetic. Call **printArray** before and after calling **doubleValues**
- **Important:** use only ***(p + i)** or pointer increment **(p++)**, never **p[i]**
- **Expected output:**
  - Before: 85 92 70 88 95
  - After: 170 184 140 176 190
- **Hint:** remember that the array name is a pointer to the first element. In **doubleValues**, use a local pointer copy so you do not lose the array start address

Exercise 4: Swapping Values Using Pointer Parameters

- Write three versions of a swap function and compare their behavior:
  - **swap1(int a, int b)** pass-by-value (does NOT swap)
  - **swap2(int* p1, int* p2)** pass pointer by value (DOES swap values)
  - **swap3(int& a, int& b)** pass-by-reference (DOES swap)
- In **main()**, declare **int x = 10, y = 20**. Call each function and print x and y before and after each call
- **Expected output:**
  - swap1 - Before: x=10 y=20 | After: x=10 y=20
  - swap2 - Before: x=10 y=20 | After: x=20 y=10
  - swap3 - Before: x=10 y=20 | After: x=20 y=10
- **Hint:** call **swap2** with **swap2(&x, &y)**. Inside **swap2**, use ***p1** and ***p2** to access and modify the original variables

Exercise 5: Array Statistics with const Pointer Parameters

- Given the array **int grades[6] = {72, 85, 90, 61, 78, 95}**, implement the following three functions, each taking a **const int*** pointer parameter and the array size:
  - **int findMin(const int* p, int size)** — returns the minimum value
  - **int findMax(const int* p, int size)** — returns the maximum value
  - **double computeAvg(const int* p, int size)** — returns the average
- Use pointer arithmetic (not array indexing) inside each function. The **const** keyword must prevent any modification of the array data — try modifying an element to see the compiler error
- **Expected output:**
  - Min: 61 Max: 95 Average: 80.17
- **Hint:** initialize min/max with ***p** (first element). With **const int***, you can read ***p** but not write ***p = value**. Cast to **double** for the average calculation